

Azure SPI: The First Mental Model

1. The familiar contract and the new responsibility

An engineer who already knows OSDU has a useful starting point: services expose familiar APIs, data partitions supply context, and applications depend on those services behaving consistently. Moving into Azure SPI does not mean learning an unrelated data platform. It means understanding the implementation, the environment around it, and the engineering work that keeps that implementation current.^{1, 2}

Consider a partition lookup for **opendes** in a development environment named **dev1**. The client asks for the partition's stored configuration. A service must answer that request, some implementation must retrieve the configuration, and an environment must supply the resources that make the implementation work. If the implementation has a defect, someone must change the right source, build it, and establish what the change does when it runs.

Those are connected responsibilities. They explain why this system has both a deployment project and an engineering project. The deployment project makes a useful Azure environment reproducible. The engineering project keeps Azure service code maintainable while the OSDU community continues changing its shared code. A service repository is where those shared and Azure-specific sources meet.

There are three meanings of **SPI** in this material:

- **The Service Provider Interface** is an internal code boundary. Common OSDU service code calls an interface; an Azure implementation provides the behavior behind it.
- **The SPI Stack** is the deployment project, **osdu-spi-stack**, and the Azure development and test environment it creates. Its command-line tool is named **spi**.
- **The SPI engineering system, osdu-spi**, is the template and workflows used to synchronize, integrate, build, and validate service forks.

An interface is a contract in code. A stack is somewhere the code runs. An engineering system is how changes to the code are handled. These meanings are related, but they describe different things. In particular, the interface is not an extra server that an OSDU request visits.

The central story has two directions. First, follow a familiar request into a running service and through its Azure provider. Then follow a provider change out to its repository, through a build, and back into an environment where it can be tested. The first direction explains behavior. The second explains responsibility and evidence.

Understanding this round trip is the purpose of orientation. The exact commands, workflow schedules, and configuration fields become useful after the objects and their relationships are familiar. An engineer can then approach the detailed documentation with a specific question rather than having to discover the entire structure while reading a procedure.

2. One service connects three repositories

The repositories are easiest to distinguish by the things they produce. **osdu-spi-stack** contains the deployment tool, Azure infrastructure definitions, and Kubernetes workload configuration. It creates and configures environments. **osdu-spi** supplies reusable engineering workflows. A service fork such as **osdu-spi-partition** contains the service source and receives those workflows. Its builds produce the partition service image.^{1, 2, 3}

The template does not contain every service's Azure implementation. That implementation belongs in the corresponding service repository. Conversely, a partition provider fix is not made in the stack's infrastructure definitions. The stack determines how a service runs; the service repository determines the code that runs.

The OSDU community supplies shared service code. The Azure service fork keeps that shared code alongside the Azure provider it maintains. A build packages them into one container image. The image is a deployable artifact; the repository is the source and history from which it was built.

The engineering template provides common machinery because every service fork needs many of the same operations. A change to that machinery can be delivered to forks as a reviewable update. This separates a workflow improvement, which can help many services, from an implementation fix, which belongs to one service.

There is also a distinction between using OSDU and maintaining a provider. An engineer can create or connect to a stack and call OSDU APIs without changing a service fork. Synchronization and local build work in a fork can also happen without an Azure environment. Live acceptance testing is where a credentialed workflow needs the running stack.

For the example, keep four concrete objects in view: the **partition repository**, the **partition image**, the **partition workload**, and the **partition configuration**. They are not interchangeable. A source change does not immediately alter a workload. An image sitting in a registry does not prove it is running. A running service must still reach the configuration needed to answer the API.

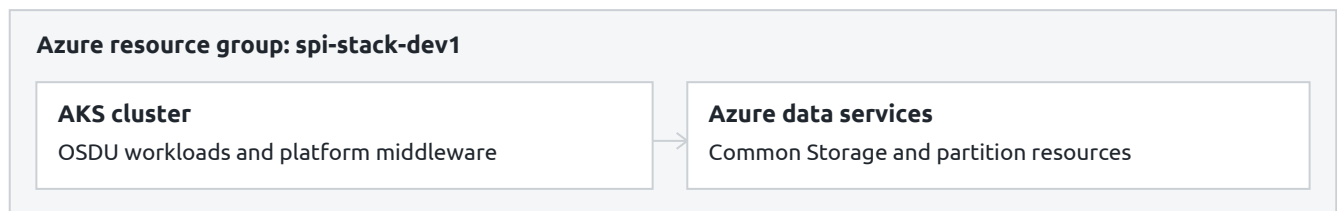
A useful way to explain the relationship is to follow a single artifact. A provider fix starts as source in the partition repository. A build turns it into an image. The stack's image selection identifies the image that should run. Kubernetes runs that image as a workload. An API result supplies evidence about its behavior.

This distinction helps route engineering work. Change service behavior in its owning source. Change shared engineering machinery in the template. Change environment creation or workload configuration in the stack project. The detailed contribution rules build on these three homes.

3. The stack is larger than the cluster

Outside the deployed stack

Workstation and source repositories



The cluster and Azure data services share the environment; source and workstation remain outside it.

The running environment named **dev1** occupies the resource group **spi-stack-dev1**. An Azure subscription is the broader account boundary in which that resource group lives. The resource group contains an AKS cluster and supporting Azure resources. **AKS**, Azure Kubernetes Service, is the cluster boundary; it is one part of the complete stack.¹

The cluster and the external Azure data services sit beside each other within the environment. Cosmos DB, Storage, and Service Bus are not hidden inside Kubernetes. A workload in the cluster reaches the Azure resources it needs through their APIs.

Inside AKS, namespaces group workloads. The **osdu** namespace contains the OSDU services. Platform middleware includes Redis, Elasticsearch, and PostgreSQL. Operators manage some of those systems. A namespace is a Kubernetes grouping; it is not an OSDU data partition.

The names **dev1** and **opendes** answer different questions. **dev1** identifies the environment. **opendes** identifies the data partition in the running example. A partition has its own Cosmos DB SQL account, Storage account, and Service Bus namespace in this stack. Other resources, including common Storage and the entitlements graph, are shared by the environment.

For the partition lookup, the important backend is **Table Storage in common Storage**. The lookup returns stored configuration describing **opendes**. It does not read an OSDU record from that partition's Cosmos DB, fetch a file from its blob storage, or send a Service Bus event. Other OSDU operations use those resources. Knowing where a resource appears on a map does not mean every request visits it.⁴

The workstation is outside this Azure environment. An engineer uses the spi CLI there to create or inspect a stack. The source repositories are also outside the deployed stack. Their contents are inputs to builds and deployment, not runtime services through which an API request travels.

The stack uses real Azure services because the behavior being developed includes Azure integration. A substitute database can be useful for a limited test, but it cannot establish that the provider works against Azure's identity, data, and messaging interfaces.

This is deliberately a **development and test environment**. It shares a workload identity across OSDU services and does not provide the backup, disaster recovery, or service-by-service Azure access isolation expected of a production design. Reproducible testing is its purpose. A complete development environment and a production operating model are different commitments.

1. OSDU SPI Stack architecture | 4. Reference partition service, revision 3a5690d; cache fix fc2dfbf

4. Creation starts several kinds of work

The command **spi up --env dev1** starts creation of the environment. It is one entry point into several kinds of work with different owners. Understanding those owners explains why the terminal returning and an API becoming usable are different events.^{5, 6}

The CLI drives **Bicep**, Azure's infrastructure definition language, to create resources and prepares the cluster's inputs. AKS is created before the Azure resources and identity bindings that depend on its identity information. The CLI also seeds configuration that the workloads will consume.

Flux is the component that applies the declared Kubernetes workload configuration. It brings up layers in dependency order. **Kubernetes controllers and operators** continue managing the things they are responsible for: running workloads, responding to failed pods, and operating middleware. The **human operator** decides when to create, inspect, deliberately update, or remove the environment.

These responsibilities overlap in time. Flux can still be assembling workloads or initialization jobs after the CLI exits successfully. Initializing partition configuration and the other prerequisites for OSDU is work in its own right. A process can exist before it can answer the request an engineer actually cares about.

There is therefore no single green light that proves every layer. The CLI's successful exit describes its orchestration. Workload readiness describes a later state. An authenticated API response supplies different evidence again. **spi status --watch** helps an operator follow the environment; an API call checks behavior at the client boundary.

The distinction between **declared state** and **observed state** is useful throughout the course. Declared state says what should run. Observed state says what is running now. Reconciliation is the continuing work that brings them together.

The stack deliberately pauses fetching new Git revisions after deployment. This keeps a later repository change from silently becoming the environment's new input. It does not stop Flux applying the revision it already has, and it does not stop Kubernetes controllers. A stable source revision can coexist with continuing work inside the cluster.

For the partition example, the order of reasoning is concrete: an environment must exist, the partition service and its backing configuration must be ready, and the caller must be admitted before the lookup can answer. A successful creation command alone does not establish all three.

Removing the environment is another deliberate operation. Ordinary **spi down** removes compute and application data while retaining identity and naming information. Rebuilding can reuse that footprint; it does not bring deleted application data back. Creation, readiness, and teardown are distinct states with distinct evidence.

5. Inside the partition service

The client calls **GET /api/partition/v1/partitions/opensdes**. The request reaches the gateway and then the partition service. In the service's code, common OSDU code calls **IPartitionService.getPartition**. The Azure implementation supplies that operation. These parts are packaged in the same service image; the interface call is not a network hop.⁴

The interface also differs from the public HTTP API. The HTTP API is the contract used by the client. The provider interface is the contract used by the common service code. A change to an internal interface can require an Azure implementation change even if the public API has not changed.

The partition provider first asks its cache for opensdes. On a miss, it reads stored configuration from Table Storage in common Storage and returns that configuration. The cache is an optimization. The stored partition configuration is the durable answer.

There is a real fix that makes this example worth following. Commit **fc2dfbf**, dated 30 July 2026 in the reference partition repository, added guarded cache operations and regression tests. When a cache read throws, the provider logs a warning and treats the failed read as a miss. It can then obtain the answer from Table Storage. A failed attempt to write the answer back to the cache is also logged rather than allowed to turn a successful durable read into a failed request.⁴

Picture the difference while the cache is unavailable. Before the guarded behavior, the cache exception could prevent the lookup from answering. With the fix, a successful table read still lets the lookup return its stored configuration. This does not make every failure harmless: an unavailable durable store or a missing partition still matters.

The fix illustrates a precise engineering principle: recover from a failure when the system knows a valid alternative, and make that recovery observable. A warning about a failed cache is useful evidence. Quietly pretending a required check succeeded would be a different behavior with a different risk.

This is why unit tests remain valuable. They can establish that a cache exception follows the fallback path and that cache failures do not erase an otherwise valid result. Live acceptance testing asks another question: does the assembled service behave through its APIs in the real environment?

The cache fallback is real source history. The later acceptance journey in this orientation is an **illustrative use of the newer template's test lane**. At the reviewed reference revision, the partition fork had not adopted that lane or written its service descriptor. The illustration explains the lane's contract; it is not a claim that this historical fix completed such a run.

4. Reference partition service, revision 3a5690d; cache fix fc2dfbf

6. The fork keeps two responsibilities together

Upstream OSDU continues developing shared service code. The Azure fork must keep receiving those improvements while maintaining its own Azure implementation. The design also anticipates upstream removing Azure provider source. Freezing a copy would gradually lose improvements; allowing upstream ownership to decide the fate of the Azure implementation would lose code the Azure service still needs.³

Ownership therefore runs through the repository. Shared modules are upstream-derived.

provider/partition-azure and the corresponding Azure test source are fork-owned. A single commit can contain both kinds of source without giving them the same maintenance rules.

The fork uses three branches with distinct jobs:

- **fork_upstream** is generated input derived from upstream. The generated tree excludes the fork-owned Azure implementation.
- **fork_integration** is the workspace where generated shared code is combined with the fork's current source.
- **main** is the protected, reviewed result used as the base for ongoing work.

The important fact is what is absent from generated input. Because the Azure provider is excluded from that input, a future upstream deletion of its own Azure directory does not instruct the fork to delete its maintained provider. Preserving source ownership is a structural property of generation, rather than a daily act of remembering which files to rescue.

That protection does not guarantee compatibility. Upstream can change a common interface in a way that requires an Azure code change. Integration and tests still have real work to do. The system protects the location where the provider is maintained; engineers remain responsible for making the implementation work with new shared code.

The integration process first brings the current main branch into the workspace, then combines the generated upstream input with it. The Azure fix is therefore evaluated with the new shared code. A reviewed integration change can then reach main. An integration PR and a later PR proposing a version release have different purposes.⁷

The template supplies the recurring workflows that perform this work and expose problems for review. It does not replace engineering judgment. If generation encounters a source path it cannot classify, it stops for a decision. That failure is useful because guessing could silently take ownership of the wrong thing.

This strictness is compatible with the cache fallback. An unrecognized source path has no established safe handling rule. A cache failure has an explicit alternative in the durable store. Good automation distinguishes those cases and makes its decisions inspectable.

3. ADR-038: Upstream filter transform | 7. Cascade integration

7. A built image is a candidate, not a conclusion

The service build packages shared code and the Azure implementation into an image. A registry holds that artifact. Its **digest** identifies the particular image being considered. A tag is a readable name; the digest is what lets a run identify the exact candidate it pins and checks.⁸

Source, image, and running workload are three separate stages. A commit identifies source. A build produces an image from that source and its build inputs. The workload must actually run the intended image before an API result can be attributed to that candidate.

Validation can build and test without a release already existing. Eligible same-repository pull requests and eligible branch pushes can enter the deployment lane when the required inputs and onboarding are present. A version release is a separate event. Acceptance testing does not inherently wait for a version tag.⁹

A **gate** decides whether a particular workflow run can enter the live lane. Eligibility, a published image, environment onboarding, and a descriptor all matter. When requirements are absent, the lane can skip with a reason. A green overall validation summary can therefore coexist with no live acceptance run.

The **service descriptor**, named `.spi/service.yaml`, belongs to the service repository. It declares the test suites and the inputs they need. The stack publishes facts about the environment. The template's machinery binds the declared needs to those facts for the run. This separates service-specific testing knowledge from environment-specific addresses and identity values.

For the running example, the guarded cache behavior has focused regression-test evidence. A live suite then supplies evidence about the API behavior it actually exercises on the pinned image. Unless that suite also deliberately exercises cache failure, its success is not a second demonstration of the fallback itself.

This distinction prevents a common misunderstanding: more green indicators do not automatically mean a broader claim was tested. Each indicator answers a particular question. Did the build succeed? Did the focused tests cover the fallback? Did this run borrow the environment? Did the intended digest become live? Did the declared API tests execute and pass?

The useful habit is to match a conclusion to its evidence. A build artifact proves an image exists. Workload verification connects the running image to the intended digest. Test reports describe observed behavior. An engineer needs that chain before saying what the candidate has demonstrated.

8. ADR-033: GHCR as the service image registry | 9. Fork deployment loop

8. Borrow, prove, restore

A standing stack lets an eligible fork run test a candidate without provisioning a new environment for every change. The run borrows a service's place in the environment for a limited test. The rest of the stack supplies the surrounding services and real Azure dependencies.^{9, 10}

Suppose the partition service normally runs image **A** and a workflow wants to test candidate **B**. A and B stand for image digests. The run records B in **osdu-image-lock**, a Kubernetes ConfigMap used by Flux to select service images. It also records ownership of its temporary pin and the canonical image that should be restored.

This lock write is the deployment input. Flux reconciles it and Kubernetes rolls out the selected image. The write and the finished rollout are separate events. The workflow verifies the expected image in the workload before using the API test results as evidence about that candidate.

The run then executes the suites declared by the service descriptor. Reports must establish that tests actually ran and passed. A successful process that ran no meaningful tests is not sufficient acceptance evidence.

After the test attempt, the workflow attempts restoration. If it still owns the temporary pin, it writes the recorded canonical image back. Flux reconciles again. A successful test on a push does not make candidate B the new permanent image by itself; canonical image advancement is a separate environment decision.

Ownership matters when circumstances change. If a newer run owns the pin, the older run leaves it alone. Writing its old restore target over another run's work would be the wrong cleanup. A lost runner can also leave a pin behind, so restoration is an operation to inspect and recover, not an unconditional promise.

Sharing an environment is also not the same as isolating every test. Runs for a service are serialized by the workflow, but different services still interact. A candidate partition service that gives incorrect answers can affect a sibling service's tests even if that sibling's image never changes.

For the example, follow the state rather than the green checkmark: A is the normal image; B is pinned; the workload is observed running B; suites produce reports; restoration either returns the owned pin to A or leaves another owner's pin untouched.

The cache fix remains the same source change throughout this journey. What changes is the kind of evidence available about it. Ownership protects where it is maintained. The build identifies an artifact. The live run tests declared behavior in an assembled Azure environment.

9. Fork deployment loop | 10. ADR-041: Borrow, prove, restore lane

9. Identity answers separate questions

There are three identity relationships worth recognizing before studying their mechanics: a **client calling OSDU**, a **service calling Azure**, and a **workflow changing the test environment**. Success in one relationship does not establish success in the others.^{11, 12}

For an incoming API request, the system must establish who the caller is and whether the service admits that caller for the operation. Authentication and authorization are related but separate. A valid token does not, on its own, establish that an operation is allowed.

The partition service is a useful reminder that authorization behavior is service-specific. Its reference Azure authorization path checks an app-only caller through its provider authorization implementation. It should not be explained as an entitlements lookup on every partition request. Other services use their own authorization paths.¹³

For outgoing Azure access, a workload uses **Workload Identity** to exchange a projected workload token for an Azure access token. Azure then authorizes that identity for the requested resource. The partition provider's table access is one such relationship.

This avoids relying on stored Azure data-service keys for that path. It does not mean the whole stack contains no credentials. Middleware passwords still exist, and the OSDU workloads share an Azure identity. One service's Azure access is not isolated from another's in this test design.

For deployment, an eligible workflow obtains access through an explicitly onboarded repository and a federated identity relationship. The deployment identity has scoped permissions to update the image-lock object and inspect workloads. A repository being onboarded and a human approving a job are different facts; a human approval pause is an additional environment policy.

The permission boundary is the image-lock object, not a separate authorization boundary around each service entry inside it. Trusted writers can affect sibling image selections. Running selected code also means giving that code the workload's effective access. Narrow deployment permissions should therefore not be described as making an untrusted service image harmless.

The orientation-level consequence is simple: this environment is shared by trusted engineering participants. Its access model supports that scope. It is not evidence of production isolation or protection against mutually untrusted service maintainers.

When an operation fails, identify the relationship first. A caller can be refused before the provider reads anything. A provider can fail to access Azure after the caller is admitted. A workflow can be unable to borrow the environment even while existing API requests work. Each failure sends the engineer to a different part of the documentation.

11. Workload Identity and request authentication | 12. ADR-032: Environment deploy identity and namespace RBAC | 13. Partition authorization source at revision 3a5690d

10. From orientation to useful engineering judgment

Return to the lookup that started the story. A client asks the partition service for opens in dev1. The request reaches a running workload. Inside the service image, common code calls an interface implemented by the Azure provider. The provider obtains stored configuration through its cache and, when needed, Table Storage in common Storage.

Now follow the change in the other direction. The cache fallback belongs to the partition fork's Azure source. The engineering system keeps that source separate from generated upstream input, combines it with current shared code, and supplies build and validation workflows. A build produces a candidate image. An eligible acceptance run can borrow the stack, verify the candidate, collect suite evidence, and attempt ownership-aware restoration.

The same service connects the two directions. The repository is where engineers change it; the stack is where the assembled behavior is observed. That connection explains why learning only Azure architecture or only the branch workflow leaves an incomplete picture.

Several principles now have concrete meaning. **Boundaries locate responsibility.** The workstation, source repositories, cluster, Azure resources, and provider implementation each answer a different part of a problem. **Ownership protects change.** Shared code and Azure code can live in one repository while obeying different maintenance rules.

Declared state needs observation. A command, a lock write, and a running image are not the same event. **Evidence has a scope.** Unit tests, deployment verification, and acceptance suites justify different conclusions. **Recovery has a contract.** A known durable fallback can preserve a request; cleanup must respect the current owner of a shared pin.

Those principles are useful without memorizing every resource, branch rule, or command. If the cache fails but the table still answers, look at provider behavior. If the new shared interface no longer compiles with Azure code, look at integration. If a job is green but never borrowed the stack, look at the gate and test evidence. If the candidate did run and the service is back on its canonical image, that may be the expected outcome of a temporary test.

The training site's first two views explore the environment and its lifecycle. The third opens one service. The fourth and fifth explain source ownership and the fork's workflow. The sixth follows a candidate back through the image lock. The final collection of misconceptions can then be used as reference when a familiar assumption does not fit.

The detailed repository guides supply the next level: exact setup, workflow behavior, identity configuration, and recovery procedures. Orientation supplies the structure needed to choose the right guide and understand why its details matter.

Sources and scope

This orientation is grounded in the source snapshots reviewed on 12 September 2026: osdu-spi-stack at dc2c956, osdu-spi at 080f0b8, and osdu-spi-partition at 3a5690d. Linked main-branch documents can evolve. The named cache fix is historical source evidence; the later partition acceptance run is illustrative because the reference fork had not adopted the newer lane. No live environment state is asserted.

1. Azure. **OSDU SPI Stack architecture**. The environment, resource boundaries, workloads, and development/test scope. [Read source](#).
2. Azure. **Engineering-system architecture**. The template's role and service-fork machinery. [Read source](#).
3. Azure. **ADR-038: Upstream filter transform**. Source ownership, the generated tree, and preservation of the Azure provider. [Read source](#).
4. Azure. **Reference partition service, revision 3a5690d; cache fix fc2dfbf**. PartitionController, IPartitionService, AuthorizationFilter, Azure AuthorizationService, PartitionServiceImpl, and the fix's regression tests. [Read source](#).
5. Azure. **Deployment lifecycle**. Creation, bootstrap, readiness, and teardown responsibilities. [Read source](#).
6. Azure. **Flux reconciliation**. Dependency ordering, source suspension, and continuing reconciliation. [Read source](#).
7. Azure. **Cascade integration**. Combining main and generated upstream input in the workspace. [Read source](#).
8. Azure. **ADR-033: GHCR as the service image registry**. Published service artifacts and image identity. [Read source](#).
9. Azure. **Fork deployment loop**. Eligibility, descriptor, pinning, verification, suite evidence, restoration, and implemented limits. [Read source](#).
10. Azure. **ADR-041: Borrow, prove, restore lane**. The template's acceptance workflow and evidence contract. [Read source](#).
11. Azure. **Workload Identity and request authentication**. Incoming callers, outgoing Azure access, and middleware credentials. [Read source](#).
12. Azure. **ADR-032: Environment deploy identity and namespace RBAC**. Repository onboarding, scoped permissions, and the shared lock's trust boundary. [Read source](#).
13. Azure. **Partition authorization source at revision 3a5690d**. AuthorizationFilter delegates to the provider; the Azure AuthorizationService checks the principal type and issuer. [Read source](#).